MCPO: Metamorphic-Causal Prompt Optimization for LLM-based Multi-Agent Systems

Weiqiang Jin[1][0000−0002−6656−6061], Tingting Sun[1], Shixiang Tang[1], Yang Liu[1], Yanghao Zhou[2], Ningwei Wang[1], and Biao Zhao[1][0000−0002−3651−0702] (✉)

[1] School of Information and Communications Engineering, Xi'an Jiaotong University, Xi'an, China
{weiqiangjin,suntt21,shixiangtang,liu0725,ningweiwang}@stu.xjtu.edu.cn, biaozhao@xjtu.edu.cn

[2] Department of Electrical and Computer Engineering, National University of Singapore, Singapore
yangzmorty132@gmail.com

Abstract. Large language model (LLM)-based multi-agent systems (MAS) are increasingly used for complex reasoning and coding tasks, where multiple specialized agents interact through role-specific prompts. However, jointly optimizing prompts remains challenging. Methods that optimize against a fixed calibration set may overlook brittleness to semantics-preserving rewrites and peer variation, while reward signals can misattribute errors in multi-step collaboration. To address this issue, we propose MCPO, a Metamorphic-Causal Prompt Optimization framework for LLM-based multi-agent systems. MCPO expands each calibration item into a set of counterfactual collaboration worlds, audits collaboration invariants across these worlds, localizes failures through causal message transplantation, and applies taxonomy-guided local prompt surgery under a held-out patch gate. In this formulation, calibration data is used not only to score prompts, but also to expose instability and convert failure traces into targeted repairs. Experiments on six reasoning and coding benchmarks show that MCPO improves end-task performance under two worker-evaluator configurations. Under a matched DeepSeek-v4-flash/DeepSeek-v4-pro setting, it improves the six-task average by 6.25 points over Hierarchical MAS + MASPO. Under a harder Qwen3-8B/Gemini-2.5-pro setting, it improves five of six datasets, raises the six-task average by 3.15 points, and yields smaller held-out rewrite gaps. These results suggest that robustness-oriented diagnosis and localized repair can be effective components of multi-agent prompt optimization.

Keywords: Multi-agent systems · Prompt optimization · Metamorphic testing · Causal attribution · Robustness

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2027

1 Introduction

Large language model based multi-agent systems (MAS) are now a standard scaffold for mathematical reasoning, science QA, and code generation [1,2]. In these systems, the per-agent system prompt is a central design variable: it defines each agent's role, shapes what evidence is preserved, and determines how reliably the resulting message can be consumed by downstream peers. As MAS pipelines grow deeper, small prompt deficiencies can propagate through the interaction graph. An omitted numerical constraint, an over-compressed rationale, or an ambiguous handoff instruction can propagate across multiple stages of the interaction graph and appear only as a final-answer failure several steps later.

Recent optimizers such as MASPO [3], MAPRO [4], MASPOB [5], and MASS [6] therefore cast prompt design as a search problem. They differ in search strategy and credit decomposition, but they still share the same operational premise: a small calibration set is treated as a static reward source, and the optimizer searches for prompts that score best on that set. This premise has led to substantial empirical gains, yet it may also introduce a source of brittleness. With only a few dozen calibration examples, reward estimates are noisy; prompts that appear effective on the original calibration surface form can be fragile to harmless rewrites; and in multi-step graphs the agent whose local reward drops the most is often not the agent that caused the failure in the first place.

These observations suggest that the challenge lies not only in credit assignment, but also in how calibration data is utilized. Furthermore, in Section 2, we provide evidence for two failure modes of reward-driven optimization: reward-driven optimizers leave a hidden robustness gap under held-out rewrites and peer-style perturbations, and reward decomposition can mis-localize blame in controlled injections. These observations suggest that calibration examples should be interrogated as robustness probes rather than treated solely as sources of scalar rewards.

Motivated by these observations, we propose Metamorphic-Causal Prompt Optimization (MCPO) for LLM-based Multi-Agent Systems. Instead of scoring a calibration item once, it expands the item into a bundle of counterfactual collaboration worlds, including input rewrites and peer-style swaps, and then asks whether the MAS remains stable across those worlds. When a violation appears, MCPO performs causal message transplantation to test which agent's message would have rescued the failure, classifies the trace into one of eight failure classes, and edits only the identified module or edge contract. A held-out gate keeps only edits that lower violation rate without unacceptable regressions on clean items.

Across six benchmarks and two LLM configurations, results indicate that MCPO improves average performance and reduces held-out rewrite gaps. Under a matched DeepSeek-v4-flash/DeepSeek-v4-pro stack, MCPO improves over Hierarchical MAS + MASPO by 6.25 points on average. Under the harder Qwen3-8B/Gemini-2.5-pro stack, the method still improves five of six datasets, raises the six-task average by 3.15 points, narrows held-out rewrite gaps, and exposes

interpretable failure traces such as round-wise invariant-violation curves and class-specific patch profiles. These diagnostics provide additional insight into the optimization process by exposing the types of failures being repaired.

Our contributions are summarized as follows.

- We introduce a new view of multi-agent prompt optimization in which the calibration set becomes a counterfactual stress-test suite rather than a static reward source.
- We propose causal message transplantation, together with taxonomy-guided targeted repair, as a message-level mechanism for turning stress-test failures into actionable prompt edits.
- We show that MCPO improves performance in a matched worker-evaluator setting, while also surfacing robustness, convergence, and failure-analysis signals that reward-driven optimizers do not expose.

2 Motivating Observations

Before introducing MCPO, we test the two empirical claims that motivate it. The first is that reward-driven prompt optimizers leave a robustness gap that is invisible on the original calibration form. The second is that, in multi-step pipelines, reward-based attribution often edits the wrong agent.

Observation 1: reward-driven optimization leaves a hidden robustness gap. We run MASPO [3] and MAPRO [4] on MATH-500, GPQA-Diamond, and HumanEval-ET under the Qwen3-8B worker and Gemini-2.5-pro evaluator setting. For each method we evaluate the final prompts on the original test set and on a held-out paraphrased counterpart produced by a disjoint rewriting prompt with the same gold answers. The two evaluations differ only in surface form.

Both optimizers lose 4–7 points under held-out rewrites. The largest degradation is observed on GPQA-Diamond, where MASPO falls from 54.04 to 47.47. The drop remains non-trivial even on MATH-500, where the benchmark is relatively templated. Figure 1(a) summarizes this pattern and shows that the trend is consistent across datasets rather than being confined to a single benchmark.

We also add a second stress test that changes only the downstream verifier style while keeping the task input fixed. Figure 1(b) shows that reward-driven prompts are again brittle, especially on GPQA-Diamond and HumanEval-ET. The fragility is therefore not only lexical; it also reflects unstable message interfaces between agents.

Observation 2: reward decomposition can mislocalize blame in controlled injections. We next construct a controlled injection audit on AQuA and MBPP with a three-agent reflect pipeline. On AQuA we inject an upstream bug that suppresses numerical constraints in the predictor. On MBPP we inject a reflector bug that marks every plan as incorrect. The injection site is therefore known. Each dataset contributes 50 injected cases, and the reported scores average over four

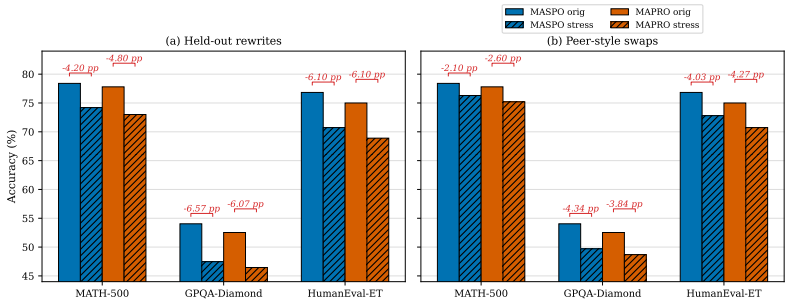


Fig. 1. Preliminary stress tests under the Qwen3-8B worker and Gemini-2.5-pro evaluator configuration. (a) Held-out rewrite gaps for reward-driven prompt optimizers. (b) Peer-style gaps when only the downstream verifier style is swapped.

Table 1. Mis-localized credit audit. Each cell reports blame@1 (%) against a known injected bug; parentheses show the share of attribution errors that incorrectly point downstream.

Attribution method	AQuA (predictor bug)	MBPP (reflector bug)
MASPO local reward	62.0 (84.2% downstream)	56.0 (77.3% downstream)
MCPO causal transplant	90.0 (40.0% downstream)	88.0 (33.3% downstream)

attribution rounds. We then compare MASPO's local-reward attribution with MCPO's causal transplantation on the same cases, measuring blame@1 against the true source and the share of attribution errors that point downstream. The corruption templates cover instruction inversion, constraint removal, ambiguity injection, contradictory constraints, and format-specification errors, matching the prompt-level intervention that MCPO is designed to repair.

Table 1 shows that MASPO frequently blames the wrong stage in these controlled injections, especially a downstream stage that inherits an upstream mistake. By contrast, causal transplantation evaluates attribution through explicit intervention on intermediate messages by testing whether replacing one agent's message with its successful counterpart recovers the final outcome. This intervention leads to higher localization accuracy in the injected setting. Taken together, the stress tests and the credit audit motivate the two design components of MCPO: counterfactual worlds for robustness and intervention-based localization for repair.

3 Method

3.1 MAS Setup and Prompt Factorization

We adopt the standard directed-graph view of a multi-agent system. Let $G = (V, E)$ denote a directed acyclic graph of strategic agents, where each agent $v_i \in V$ is associated with a system prompt p_i and an LLM rollout function f_i. Given a question q and the concatenated messages from its in-neighbors C_i,

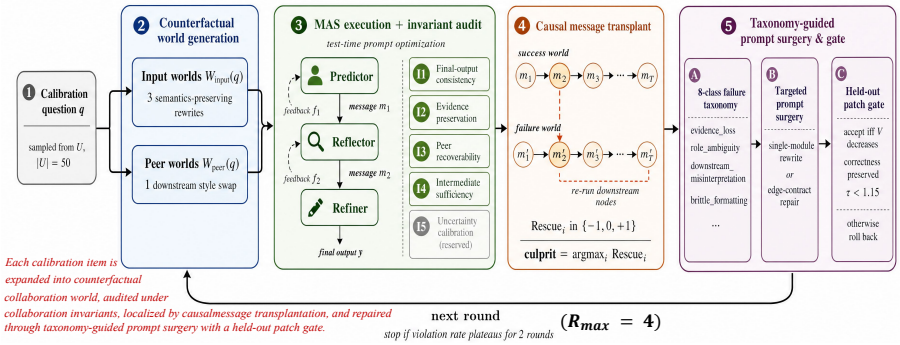


Fig. 2. Overview of MCPO. Calibration questions are expanded into counterfactual worlds, audited under collaboration invariants, localized by causal transplantation, and repaired through targeted prompt surgery.

agent i emits

$$o_i = f_i(p_i, q, C_i), \tag{1}$$

and the system returns a final answer $o_{\text{glob}} = \Phi(G, P, q)$ for $P = \{p_i\}_{i=1}^{|V|}$.

MCPO operates on a small calibration pool $U = \{q_1, \ldots, q_m\}$, with $m = 50$ in the main experiments. Rather than editing full prompts directly, it first factorizes each prompt into seven modules: `role`, `input_interpretation`, `output_contract`, and four auxiliary modules for uncertainty, downstream awareness, error handling, and final formatting. This factorization serves two objectives. First, it makes failures easier to localize semantically. Second, it constrains each update to a narrow scope, reducing the risk of performance degradation from full-prompt rewrites.

3.2 Counterfactual Worlds and Collaboration Invariants

For each calibration question q, MCPO constructs a bundle of counterfactual collaboration worlds

$$W(q) = W_{\text{input}}(q) \cup W_{\text{peer}}(q). \tag{2}$$

An input world paraphrases q while preserving answer semantics. A peer world keeps q fixed but swaps one downstream prompt for a stylistically different prompt that plays the same role. In the main loop we generate three input worlds and one peer world per question.

The optimizer evaluates whether the MAS exhibits consistent behavior across these worlds.

```
We use four active invariants:
I1 Final-output consistency. The final answers for the
    original question and its paraphrases should agree.
I2 Evidence preservation. Decision-relevant evidence in an
    upstream message should persist across paraphrases.
```

```
I3 Peer recoverability. Replacing a downstream peer with a
   stylistic alternative should not change the final answer.
I4 Intermediate sufficiency. An upstream message should
   remain sufficient for the downstream consumer to reach a
   stable conclusion.
```

An uncertainty-calibration check is implemented in the codebase but excluded from the main score to keep the budget aligned with the MASPO comparison.

Operationally, each question receives three input rewrites and one peer-style swap. World acceptance requires gold-answer agreement when labels are available and evaluator verification otherwise. I2 uses a four-point evidence rubric, I4 uses a binary sufficiency check, and code recovery is determined by unit tests before evaluator tie-breaking.

For a mini-batch $B_t \subset U$ at round t, we measure the violation rate

$$V_t = \frac{1}{|B_t|} \sum_{q \in B_t} \mathbf{1}\{\text{any invariant is violated in } W(q)\}. \tag{3}$$

This is the outer-loop signal that MCPO tries to reduce. Although the implementation is patch-based rather than differentiable, the search target can be written as a robust empirical objective

$$\mathcal{J}_t(P) = \frac{1}{|B_t|} \sum_{q \in B_t} \Big[\ell_{\text{orig}}(q; P) + \lambda \frac{1}{|W(q)|} \sum_{w \in W(q)} \ell_{\text{world}}(w, q; P) + \mu \operatorname{Var}_{w \in W(q)} s(w, q; P) \Big], \tag{4}$$

where ℓ_{orig} penalizes clean-world mistakes, ℓ_{world} penalizes world failures, and $s(w, q; P)$ is the evaluator score of the final answer in world w. The variance term penalizes prompt updates whose behavior changes sharply across semantically equivalent worlds.

3.3 Causal Message Transplantation

When a question violates at least one invariant, MCPO pairs a successful world w_s with a failing world w_f for the same base item and estimates which agent most likely contributed to the violation. For each strategic agent i, we transplant its message $o_i^{(w_s)}$ from the successful run into the failing run, keep every prompt fixed, and re-run only the descendants of i. The rescue score is

$$\text{Rescue}_i(w_s, w_f) = \begin{cases} 1, & \text{if the final answer recovers,} \\ 0, & \text{if the final answer is unchanged,} \\ -1, & \text{if the final answer worsens.} \end{cases} \tag{5}$$

Across the candidate success-failure pairs $\Pi(q)$ for the same base item, MCPO aggregates evidence with

$$S_i(q) = \sum_{(w_s, w_f) \in \Pi(q)} \alpha_{w_s, w_f} \, \text{Rescue}_i(w_s, w_f), \qquad \hat{\imath}(q) = \arg\max_{i \in V} S_i(q), \tag{6}$$

Table 2. Failure taxonomy and repair target in MCPO.

Failure class	Repair target	Failure class	Repair target
`evidence_loss`	`output_contract`	`downstream_misinterpretation`	edge contract
`over_compression`	`output_contract`	`brittle_formatting`	`final_formatting`
`role_ambiguity`	`role`	`peer_overfitting`	edge contract
`uncertainty_collapse`	`uncertainty`	`hallucinated_handoff`	`error_handling`

where α_{w_s,w_f} upweights pairs that violate more invariants. The identified source of failure is the agent with the largest aggregated rescue score. Since our judges operate on pairwise comparisons rather than isolated scalar ratings, this step follows a pairwise-comparison paradigm similar to that used in LLM-as-a-judge studies [7].

3.4 Taxonomy-Guided Surgery

Once the source of failure is identified, MCPO assigns the failure to one of eight classes and edits only the corresponding module. Table 2 shows the routing dictionary.

The classifier receives the violated invariant, the identified agent's success and failure messages, and the failing downstream trace. Edge repair is used only when the failure label explicitly indicates a handoff mismatch; otherwise MCPO rewrites one local module and leaves the rest untouched. This design is motivated by the observation that many prompts already contain useful task-specific instructions, whereas a small calibration set rarely supports reliable global rewrites.

Each candidate patch is then evaluated on a held-out subset $H_t \subset U \setminus B_t$. We accept a patch only if it lowers violation rate, preserves held-out correctness up to one example, and keeps token overhead under $1.15\times$:

$$\text{ACCEPT} \iff \tilde{V}_t < V_t \ \wedge \ \tilde{C}_t \geq C_t - 1 \ \wedge \ \tau < 1.15. \tag{7}$$

Rejected patches are rolled back immediately.

3.5 Theoretical Justification

Theorem 1 (single-fault identifiability). Assume a success world w_s and a failure world w_f differ at exactly one agent c, all ancestors of c emit the same messages in both worlds, and every descendant agent is a deterministic function of its prompt and parent messages. Then transplanting $o_c^{(w_s)}$ into w_f and re-running descendants reproduces the success-world downstream trace, whereas transplanting any non-ancestor-preserving message $o_j^{(w_s)}$ for $j \neq c$ cannot remove the first causal discrepancy. Hence $\hat{\imath}(q) = c$ under the rescue-score rule.

Sketch of proof. The first causal discrepancy on the path to the final answer is $o_c^{(w_f)} \neq o_c^{(w_s)}$. Replacing it aligns every descendant input with the success world, so deterministic re-execution recovers the same downstream trace and final answer. Replacing any other message leaves that first discrepancy intact.

Proposition 1 (monotonic gated progress). Define the held-out merit function

$$\mathcal{M}_t(P) = C_t(P) - \lambda V_t(P) - \eta \max(0, \tau(P) - 1.15), \tag{8}$$

with $\lambda \geq |H_t|$ and $\eta > 0$, where $C_t(P)$ is the number of correct held-out items under prompts P. Any accepted patch $P \to P'$ satisfies $\mathcal{M}_t(P') > \mathcal{M}_t(P)$.

Sketch of proof. Acceptance enforces $\tau(P') < 1.15$, $C_t(P') \geq C_t(P) - 1$, and $V_t(P') < V_t(P)$. Since V_t is an empirical mean over $|H_t|$ items, any strict decrease is at least $1/|H_t|$, so

$$\mathcal{M}_t(P') - \mathcal{M}_t(P) \geq -1 + \lambda\big(V_t(P) - V_t(P')\big) \geq -1 + \lambda/|H_t| \geq 0, \tag{9}$$

with strict inequality whenever correctness is preserved exactly or more than one held-out violation disappears.

Optimization loop. In each round, MCPO samples a mini-batch, generates worlds for each item, measures invariant violations, localizes each violating case by transplantation, proposes one targeted repair, and keeps the repair only if it passes the held-out gate. We use $R = 4$ rounds, three input worlds, one peer world, mini-batch size 6, hold-out size 4, and one success-failure transplant pair per violating item. This budget stays within a small constant factor of MASPO under the same worker-evaluator split.

4 Experiments

4.1 Experimental Setup

Benchmarks. We evaluate on six benchmarks spanning math, science QA, and code: MATH-500 [8], AGIEval-MATH [9], AQuA [10], GPQA-Diamond [11], MBPP [12], and HumanEval-ET [13]. Every accuracy cell is reported as round(correct/$N \times 100, 2$).

Backbones and topology. We use two backbone configurations. *Qwen+Gemini* adopts Qwen3-8B as the worker model and Gemini-2.5-pro as the evaluator. *DeepSeek* adopts DeepSeek-v4-flash as the worker and DeepSeek-v4-pro as the evaluator. The default system is the three-agent hierarchical reflect topology from MASPO with reflect depth two.

Baselines and budget. Under Qwen+Gemini we compare against Vanilla, CoT [14], SC(CoT) [15], Self-Refine [16], AgentDropout [17], Hierarchical MAS + SPO, and Hierarchical MAS + MASPO [3]. Under DeepSeek we report the strongest comparable subset ending with MASPO. All methods share the same 50-example calibration pool. MCPO uses four rounds, three input worlds, one peer world, a mini-batch of six, and a hold-out of four.

Table 3. Accuracy (%) under the Qwen3-8B worker with Gemini-2.5-pro evaluator configuration.

Method	MATH-500	AGIEval	AQuA	GPQA	MBPP	HumanEval-ET	Avg_6
Vanilla	74.80	55.86	79.53	45.96	63.47	71.95	65.26
CoT	75.40	54.69	81.89	46.72	64.17	72.26	65.86
SC(CoT)	75.50	56.64	82.17	47.52	64.49	72.56	66.48
Self-Refine	76.20	56.52	82.28	47.73	64.17	70.73	66.27
AgentDropout	76.80	59.77	86.23	47.98	58.09	65.86	65.79
Hier. MAS + SPO	77.80	63.41	86.61	51.01	61.83	73.39	69.01
Hier. MAS + MASPO	78.40	64.45	87.01	54.04	65.34	76.83	71.01
MCPO	**85.80**	**66.33**	**93.58**	**58.54**	61.91	**78.78**	**74.16**
(Δ vs. Hier. MAS + MASPO)	+7.40	+1.88	+6.57	+4.50	-3.43	+1.95	+3.15

Table 4. Accuracy (%) under the DeepSeek-v4-flash worker with DeepSeek-v4-pro evaluator configuration.

Method	MATH-500	AGIEval	AQuA	GPQA	MBPP	HumanEval-ET	Avg_6
Vanilla	55.60	38.67	76.38	32.32	60.42	80.49	57.31
CoT	56.20	39.45	78.35	33.33	60.66	81.10	58.18
SC(CoT)	56.80	41.02	79.13	34.34	60.89	81.71	58.98
Self-Refine	57.40	40.62	79.92	34.85	61.36	80.49	59.11
AgentDropout	57.80	42.97	81.50	36.36	56.21	73.78	58.10
Hier. MAS + SPO	57.90	42.58	81.89	35.86	62.77	83.54	60.76
Hier. MAS + MASPO	58.20	43.36	82.28	36.87	63.70	84.76	61.53
MCPO	**69.80**	**54.30**	**84.65**	**44.44**	**66.28**	**87.20**	**67.78**
(Δ vs. Hier. MAS + MASPO)	+11.60	+10.94	+2.37	+7.57	+2.58	+2.44	+6.25

4.2 Main Results

Table 3 reports the harder cross-family setting with a Qwen3-8B worker and Gemini-2.5-pro evaluator. MCPO outperforms Hierarchical MAS + MASPO on five of the six datasets, raises the six-task average by 3.15 points, and achieves the best scores on MATH-500, AGIEval, AQuA, GPQA-Diamond, and HumanEval-ET. Performance decreases on MBPP relative to Hierarchical MAS + MASPO, suggesting that robustness-oriented repair transfers broadly across the cross-family stack but remains sensitive to code-generation mismatch.

Table 4 shows the matched DeepSeek-v4-flash/DeepSeek-v4-pro configuration. Under this setting, MCPO outperforms Hierarchical MAS + MASPO on all six datasets and improves the six-dataset average by 6.25 points, with the largest gains observed on MATH-500, AGIEval, and GPQA-Diamond.

Two observations emerge from these results. A matched worker-evaluator stack is associated with larger accuracy gains, and even the harder cross-family stack benefits when calibration data is leveraged for robustness assessment in addition to reward estimation.

4.3 Robustness and Budget

Table 5 evaluates the failure mode that motivates MCPO. On all three representative datasets, MCPO narrows the held-out rewrite gap relative to MASPO.

Table 5. Held-out rewrite robustness and optimization cost. Left: held-out rewrite robustness. Right: optimization cost under the two backbone configurations.

Dataset	MASPO			MCPO		
	Orig.	Rewrite	Gap	Orig.	Rewrite	Gap
MATH-500	78.40	74.60	3.80	85.80	84.10	1.70
GPQA-Diamond	54.04	48.48	5.56	58.54	55.81	2.73
HumanEval-ET	76.83	71.34	5.49	78.78	76.95	1.83

Cfg.	Method	Calls	Tok. (k)
Q+G	MASPO	462	438
Q+G	MCPO	504	471
DS	MASPO	454	421
DS	MCPO	492	455

Table 6. Ablation under the DeepSeek-v4-flash worker and DeepSeek-v4-pro evaluator configuration.

Variant	MATH-500	GPQA	MBPP	HumanEval-ET
Full MCPO	69.80	44.44	66.28	87.20
w/o Counterfactual Worlds	64.20 (5.60↓)	38.89 (5.55↓)	63.70 (2.58↓)	83.54 (3.66↓)
w/o Causal Transplant	66.40 (3.40↓)	40.40 (4.04↓)	65.10 (1.18↓)	85.37 (1.83↓)
w/o Single-Module Surgery	70.00 (0.20↑)	43.43 (1.01↓)	64.88 (1.40↓)	84.76 (2.44↓)

The same table reports optimization cost under both backbone configurations: calls and tokens increase slightly, while held-out rewrite gaps remain smaller.

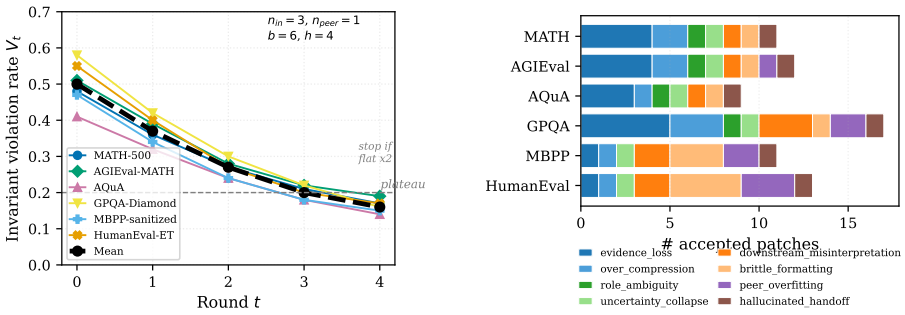
4.4 Ablation and Diagnostics

Table 6 reports three ablations on four representative benchmarks under the matched DeepSeek setting. Removing counterfactual worlds consistently hurts, removing causal transplantation also degrades performance, and replacing module surgery with broader rewrites hurts code most. The first two rows also separate the benefit of robustness-oriented data augmentation from causal diagnosis: diverse worlds help, but transplantation supplies additional repair precision beyond exposure to more rewritten examples.

Figure 3(a) plots the optimization trajectory. The mean invariant-violation rate falls from 0.51 at round 0 to 0.18 at round 4, indicating that violation rates decrease over optimization rounds. Figure 3(b) shows the accepted-patch distribution across failure classes. Evidence-preservation and communication-contract failures dominate on GPQA-Diamond and HumanEval-ET, aligning with the larger gains on those datasets.

5 Related Work

Recent work on LLM-based multi-agent systems has focused on designing collaboration patterns for reasoning and tool use. Early frameworks such as AutoGen and multi-agent debate showed that decomposing a task across interacting agents can improve reasoning performance and controllability [1,2]. Building on this direction, MASPO [3], MAPRO [4], MASPOB [5], and MASS [6] optimize prompts or agent topologies directly for multi-agent pipelines. These methods differ in search strategy and credit decomposition, but they generally optimize against a



(a) Invariant-violation rate over rounds.

(b) Accepted-patch distribution across failure classes and datasets.

Fig. 3. Optimization diagnostics. Panel (a) plots invariant-violation rate over rounds; the thick black curve is the mean across datasets. Panel (b) plots the distribution of accepted patches across failure classes and datasets.

score computed on a small calibration set. Prompt optimization in single-agent or loosely structured pipelines provides a second comparison point. OPRO [18] and DSPy [19] treat prompts or LM programs as objects to be searched or compiled against development feedback, while CoT, self-consistency, Self-Refine, and AgentDropout show how reasoning performance can depend strongly on intermediate prompting, sampling, and iterative revision [14,15,16,17].

However, these approaches do not explicitly model how errors introduced by one agent propagate through downstream message passing, nor do they localize failures through interventions on internal communications. Our method, MCPO, adapts the ideas of metamorphic testing and CheckList evaluate systems to message-level diagnosis in multi-agent systems: it perturbs input and peer worlds, audits collaboration invariants, and uses causal transplantation to estimate which prompt most likely contributed to a violation. It addresses a complementary limitation: instead of treating calibration examples only as a reward source, it uses them to construct counterfactual worlds that expose robustness failures and support localized repair.

6 Conclusion

In this work, we presented MCPO, a test-time prompt optimizer for LLM-based multi-agent systems that uses calibration data as robustness evidence rather than as a static reward source. Instead of scoring prompts on a tiny calibration set once and updating whichever agent receives the lowest reward, MCPO expands each item into counterfactual input and peer worlds, audits collaboration invariants across those worlds, localizes failures through causal message transplantation, and applies taxonomy-guided local repair under a held-out gate. This reformulates prompt optimization as an iterative diagnosis-and-repair process guided by robustness violations. Results indicate that MCPO consistently improves end-task performance over competitive baselines across multiple benchmarks and LLM configurations. In addition, the method produces interpretable

diagnostics, including round-wise violation trajectories and failure-class patch distributions, which provide insight into the optimization dynamics compared with purely reward-driven search procedures. The results suggest that effective multi-agent prompt optimization may benefit not only from improved search and credit assignment, but also from robustness-oriented use of calibration data. In the future, we plan to extend MCPO to richer agent graphs, jointly optimize prompts and communication interfaces, and learn more adaptive world generation and repair policies.

Acknowledgements. This work was primarily carried out by Weiqiang Jin at Xi'an Jiaotong University, and the corresponding author of this paper is Biao Zhao from Xi'an Jiaotong University.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C.: Autogen: Enabling next-gen LLM applications via multi-agent conversations. In: First Conference on Language Modeling (2024), https://openreview.net/forum?id=BAakY1hNKS
2. Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I.: Improving factuality and reasoning in language models through multiagent debate. In: Proceedings of the 41st International Conference on Machine Learning (2024), https://arxiv.org/abs/2305.14325
3. Wang, Z., Liu, X., Wang, L., Shan, Z., Wang, Y., Song, Z., Zhang, M.: MASPO: Joint prompt optimization for LLM-based multi-agent systems. In: Proceedings of the 43rd International Conference on Machine Learning (2026), https://arxiv.org/abs/2605.06623
4. Zhang, Z., Ge, L., Li, H., Zhu, W., Zhang, C., Ye, Y.: MAPRO: Recasting multi-agent prompt optimization as maximum a posteriori inference. In: Findings of the Association for Computational Linguistics: EACL 2026. pp. 4458–4480. Association for Computational Linguistics (2026), https://aclanthology.org/2026.findings-eacl.233/
5. Hong, Z., Zhang, Q., Sun, J., Shang, Z., Kong, M., Wang, X., Shu, Y., Dai, Z.: MASPOB: Bandit-based prompt optimization for multi-agent systems with graph neural networks. CoRR **abs/2603.02630** (2026), https://arxiv.org/abs/2603.02630
6. Zhou, H., Wan, X., Sun, R., Palangi, H., Iqbal, S., Vulić, I., Korhonen, A., Arık, S.Ö.: Multi-agent design: Optimizing agents with better prompts and topologies. In: Proceedings of the 42nd International Conference on Machine Learning (2025), https://openreview.net/forum?id=uCKvHweh1g
7. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-bench and chatbot arena. In: Advances in Neural Information Processing Systems 36: Datasets and Benchmarks Track (2023), https://openreview.net/forum?id=uccHPGDlao

8. Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., Steinhardt, J.: Measuring mathematical problem solving with the MATH dataset. In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (2021), https://arxiv.org/abs/2103.03874
9. Zhong, W., Cui, R., Guo, Y., Liang, Y., Lu, S., Wang, Y., Saied, A., Chen, W., Duan, N.: AGIEval: A human-centric benchmark for evaluating foundation models. In: Findings of the Association for Computational Linguistics: NAACL 2024. pp. 2299–2314. Association for Computational Linguistics (2024), https://aclanthology.org/2024.findings-naacl.149/
10. Ling, W., Yogatama, D., Dyer, C., Blunsom, P.: Program induction by rationale generation: Learning to solve and explain algebraic word problems. In: Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 158–167. Association for Computational Linguistics (2017). https://doi.org/10.18653/v1/P17-1015, https://aclanthology.org/P17-1015/
11. Rein, D., Hou, B.L., Stickland, A.C., Petty, J., Pang, R.Y., Dirani, J., Michael, J., Bowman, S.R.: GPQA: A graduate-level google-proof Q&A benchmark. In: First Conference on Language Modeling (COLM) (2024), https://openreview.net/forum?id=Ti67584b98
12. Austin, J., Odena, A., Nye, M., Bosma, M., Michalewski, H., Dohan, D., Jiang, E., Cai, C., Terry, M., Le, Q., Sutton, C.: Program synthesis with large language models. arXiv preprint arXiv:2108.07732 (2021), https://arxiv.org/abs/2108.07732
13. Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H.P., Kaplan, J., Edwards, H., Burda, Y., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021), https://arxiv.org/abs/2107.03374
14. Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E.H., Le, Q.V., Zhou, D.: Chain-of-thought prompting elicits reasoning in large language models. In: Advances in Neural Information Processing Systems 35 (NeurIPS 2022) (2022), https://arxiv.org/abs/2201.11903
15. Wang, X., Wei, J., Schuurmans, D., Le, Q., Chi, E.H., Narang, S., Chowdhery, A., Zhou, D.: Self-consistency improves chain of thought reasoning in language models. In: The Eleventh International Conference on Learning Representations (2023), https://arxiv.org/abs/2203.11171
16. Madaan, A., Tandon, N., Gupta, P., Hallinan, S., Gao, L., Wiegreffe, S., Alon, U., Dziri, N., Prabhumoye, S., Yang, Y., Gupta, S., Majumder, B.P., Hermann, K., Welleck, S., Yazdanbakhsh, A., Clark, P.: Self-refine: Iterative refinement with self-feedback. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023) (2023), https://arxiv.org/abs/2303.17651
17. Wang, Z., Wang, Y., Liu, X., Ding, L., Zhang, M., Liu, J., et al.: AgentDropout: Dynamic agent elimination for token-efficient and high-performance LLM-based multi-agent collaboration. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics. pp. 24013–24035. Association for Computational Linguistics (2025). https://doi.org/10.18653/v1/2025.acl-long.1170
18. Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q.V., Zhou, D., Chen, X.: Large language models as optimizers. In: The Twelfth International Conference on Learning Representations (2024), https://openreview.net/forum?id=Bb4VGOWELI
19. Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., Haq, S., Sharma, A., Joshi, T.T., Moazam, H., Miller, H., Zaharia, M., Potts, C.: DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In: The Twelfth International Conference on Learning Representations (2024), https://openreview.net/forum?id=sY5N0zY5Od